

Atividade 1: Modelagem Orientada a Objetos

POO - BSI - Ifes Serra

22 de setembro de 2026

1 Diagrama de Classes UML

A Linguagem de Modelagem Unificada (UML) ajuda a modelar sistemas de diversas maneiras. Um dos tipos mais populares na UML é o **diagrama de classes**, que descreve o que deve estar presente no sistema a ser modelado incluindo as classes, seus atributos, suas operações e seus relacionamentos com outras classes. A UML foi criada para padronizar a descrição dos diagramas OO. Um diagrama de classes permite ilustrar modelos de dados para sistemas de informação (simples ou complexos), entender melhor a visão geral dos esquemas de uma aplicação, expressar visualmente as necessidades específicas de um sistema, e fornecer uma descrição do sistema independente de implementação.

Uma classe no diagrama é representada por um retângulo dividido em três partes:

- A parte superior contém o nome da classe e é sempre necessária.
- A parte do meio contém os atributos da classe. Cada atributo é exibido em uma linha separada. Cada linha possui o nome do atributo e seu tipo, separados por um “:”.
- A parte inferior inclui as funcionalidades da classe (métodos). Cada operação é exibida em uma linha separada. O método é descrito por seu nome e sua assinatura (lista de parâmetros entre parênteses e separados por vírgula, e o tipo do retorno do método após o “:”).

Atributos e métodos possuem modificadores de acesso, que definem sua visibilidade: público (+), privado (-) e protegido (#). Exemplo:

Data
- dia : int - mes : int - ano : int
+ anterior(d : Data) : boolean

Pessoa
- nome : String - cpf : String - nasc : Data - sexo : char
+ idade(hoje : Data) : int

Métodos e atributos estáticos são representados sublinhados. Atributos estáticos em Java pertencem à classe, não a instâncias individuais, permitindo compartilhar dados entre todos os objetos. Eles são definidos com a palavra-chave *static* e acessados diretamente pelo nome da classe, sem necessidade de instanciar objetos. Existe apenas uma cópia do atributo estático na memória, independentemente do número de objetos criados. Métodos estáticos também pertencem à classe e não a objetos específicos. Eles podem ser chamados diretamente pelo nome da classe, sem que um objeto da classe precise ser instanciado e associado a ele, e não podem acessar atributos que não forem estáticos. São ideais para funções utilitárias ou compartilhadas, como `Math.sqrt()`. Exemplo:

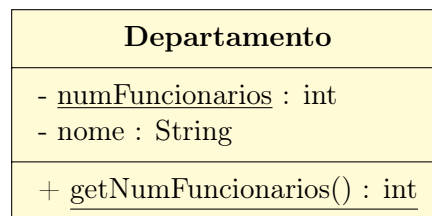
```

public class Departamento {
    // Cada departamento tem seu nome, mas o numero de funcionarios
    // eh unico para toda a empresa
    public static int numFuncionarios = 0;
    public String nome;

    public Departamento(String n) {
        this.nome = n;
        numFuncionarios++; // incrementa a variavel estatica a cada novo funcionario
    }

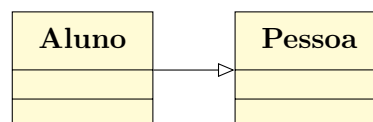
    public static getNumFuncionarios() {
        return numFuncionarios;
    }
}

```

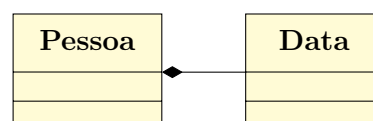


Diagramas de classe também podem exibir o relacionamento entre as classes do sistema. Nesta atividade, iremos considerar os seguintes tipos de relacionamentos:

1. A seta entre as classes Aluno e Pessoa indica **herança** entre elas. Neste caso, Aluno herda de Pessoa. Um objeto da classe Aluno também “é um” objeto da classe Pessoa, mas pode conter atributos e métodos próprios.

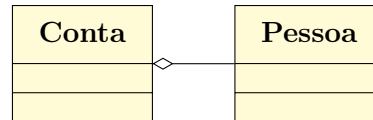


2. O losango fechado entre Pessoa e Data indica uma relação de **composição**. Neste caso, podemos dizer que a classe Pessoa “tem” um objeto da classe Data (a data em que a pessoa nasceu). Por ser uma composição, o objeto da classe **Data** vai ser instanciado no momento em que a avaliação for criada e existirá apenas ali dentro do objeto da classe **Pessoa**. Neste exemplo, o sistema não precisa armazenar uma Data sem que ela esteja associada a uma Pessoa específica. Quando o objeto da classe Pessoa for destruído, o objeto com sua data de nascimento será destruído junto. Podemos dizer que a data é uma “parte da” Pessoa.

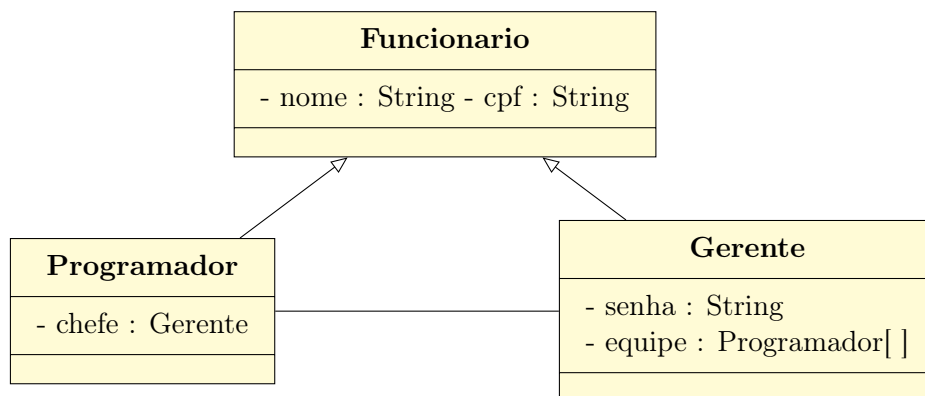


3. O losango aberto entre Conta e Pessoa indica uma relação de **agregação**. Novamente, podemos dizer que a classe Conta “tem” um objeto da classe Pessoa (o titular da conta). Entretanto, neste caso, o objeto Pessoa não será instanciado apenas quando a Conta for

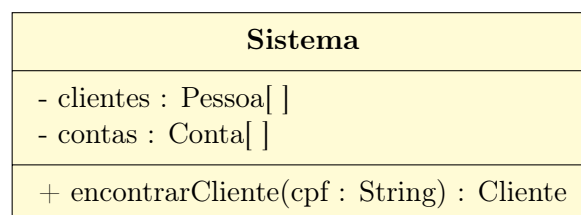
criada. A Pessoa pode existir no sistema sem a Conta existir. Quando a conta é criada, as pessoas já existiam no sistema, e a conta faz apenas uma referência a uma delas como sendo o titular dessa conta. Quando o objeto da classe Conta for destruído, seu titular continuará existindo no sistema (ele pode ter ou vir a ter outras contas no banco, por exemplo).



4. Quando houver mais de uma ligação entre duas classes, podemos representar a conexão entre elas como uma associação simples, representada apenas por uma linha conectando as duas classes (evitando a duplicidade de associações). No exemplo a seguir, um Programador possui um chefe (da classe Gerente), e um Gerente possui um conjunto de Programadores em sua equipe:



Por fim, neste trabalho, usaremos uma notação única para representar um conjunto de objetos (independente de este conjunto ser armazenado como um array, uma lista, etc). No exemplo abaixo, por exemplo, a classe sistema possui um conjunto de pessoas e um conjunto de contas:



2 O Sistema: Gestão Simplificada do ENADE

Neste ano, os alunos concluintes do curso de BSI participarão do Exame Nacional de Desempenho dos Estudantes (ENADE). Para compreender melhor a aplicação dos conceitos de Programação Orientada a Objetos, você foi encarregado de modelar um sistema simplificado para gerenciar informações relacionadas à participação dos estudantes no exame.

O sistema deve permitir o cadastro de áreas, cursos e alunos, além de armazenar as perguntas de um questionário que deve ser respondido pelos estudantes. Os alunos podem ser ingressantes ou concluintes. Todos devem responder ao questionário, enquanto os concluintes também participam da prova do ENADE.

Além de controlar a situação dos alunos, o sistema deve armazenar os resultados das provas dos concluintes e permitir o cálculo de um conceito para cada curso. Para fins desta atividade, serão utilizadas regras simplificadas para o cálculo das notas e conceitos, não sendo necessário reproduzir integralmente a metodologia oficial utilizada pelo ENADE.

3 As Classes e Atributos

O sistema deve armazenar um conjunto de **áreas**, um conjunto de **cursos**, um conjunto de **alunos** e um conjunto de **perguntas do questionário**.

Cada **área** possui um código, um nome e um conjunto de cursos pertencentes a ela. Um mesmo curso pertence a apenas uma área.

Cada **curso** possui um código, um nome e um conjunto de alunos. Os cursos armazenados em uma área e os alunos armazenados em um curso são referências aos mesmos objetos que também estão cadastrados nos conjuntos gerais do sistema.

Existem dois tipos de alunos: **ingressantes** e **concluintes**. Todos os alunos possuem nome, matrícula, um conjunto com suas respostas ao questionário e um valor lógico indicando se o questionário foi completamente respondido.

As respostas do questionário devem ser armazenadas como números inteiros. Cada número indica a posição da alternativa escolhida pelo aluno em uma determinada pergunta. Por exemplo, o valor 0 pode representar a primeira alternativa, o valor 1 a segunda alternativa, e assim por diante.

Um **aluno ingressante** possui também o semestre em que ingressou no curso. Essa informação permitirá, por exemplo, analisar posteriormente a participação no questionário de ingressantes de diferentes semestres.

Um **aluno concluinte** possui também o **resultado** de sua prova do ENADE. Esse resultado armazena a nota obtida na parte de Formação Geral, a nota obtida no Componente Específico e um valor lógico indicando se o aluno esteve presente na prova.

Cada **pergunta do questionário** possui um enunciado e um conjunto de alternativas. As perguntas não possuem uma resposta correta, pois o objetivo do questionário é coletar informações e opiniões dos estudantes.

4 Regras de Negócio para o Sistema

- *Questionário*: Existe um único conjunto de perguntas de questionário para todo o sistema. Todos os alunos, ingressantes ou concluintes e independentemente de seu curso, devem responder às mesmas perguntas.
- *Regularidade dos ingressantes*: Um aluno ingressante é considerado regular quando tiver respondido completamente ao questionário.
- *Regularidade dos concluintes*: Um aluno concluinte é considerado regular somente quando tiver respondido completamente ao questionário e estiver registrado como presente na prova do ENADE.
- *Nota da prova*: Para fins desta atividade, a nota final de um aluno concluinte será calculada a partir das notas de Formação Geral e do Componente Específico. Será utilizada a seguinte regra simplificada:

$$\text{Nota} = (\text{Nota de Formação Geral} * 0,25) + (\text{Nota do Componente Específico} * 0,75)$$

Apenas alunos presentes na prova devem ter sua nota considerada no cálculo dos resultados do curso.

- *Média do curso*: A média de um curso corresponde à média das notas finais dos seus alunos concluintes que estiveram presentes na prova.
- *Conceito do curso*: Para simplificar o sistema, o Conceito ENADE de cada curso será um número inteiro entre 1 e 5, calculado a partir da média do curso de acordo com as seguintes faixas:
 - conceito 1: média menor que 2,0;
 - conceito 2: média maior ou igual a 2,0 e menor que 4,0;
 - conceito 3: média maior ou igual a 4,0 e menor que 6,0;
 - conceito 4: média maior ou igual a 6,0 e menor que 8,0;
 - conceito 5: média maior ou igual a 8,0.

5 Funcionalidades

Um aluno deve ser capaz de registrar suas respostas ao questionário. Quando todas as respostas forem registradas, o sistema deve indicar que o questionário daquele aluno foi respondido.

Todo aluno deve também possuir um método que indique se ele está **regular** no ENADE. Para um aluno em geral, a regularidade depende apenas de ter respondido ao questionário. Para um aluno concluinte, ele precisa também ter comparecido à prova.

O resultado da prova de um aluno concluinte deve possuir um método que calcule sua nota final utilizando as notas de Formação Geral e do Componente Específico, conforme a regra especificada anteriormente.

Um curso deve possuir métodos para adicionar alunos ao seu conjunto, calcular a média das notas de seus concluintes presentes na prova e calcular seu Conceito ENADE.

Além disso, um curso deve permitir consultar seus alunos ingressantes:

- se receber um número inteiro representando um semestre de ingresso, deve retornar apenas os ingressantes daquele semestre.
- se não receber parâmetros, deve retornar todos os alunos ingressantes daquele curso;

Uma área deve possuir um método para adicionar um curso ao seu conjunto de cursos.

Uma pergunta do questionário deve possuir um método para adicionar uma nova alternativa ao seu conjunto de alternativas.

Por fim, a classe principal do sistema deve possuir as seguintes funcionalidades:

- cadastrar uma nova pergunta do questionário;
- cadastrar uma nova área;
- cadastrar um novo curso, dada a área à qual ele pertence. O curso deve ser inserido no conjunto geral de cursos do sistema e também no conjunto de cursos daquela área;
- cadastrar um novo aluno, dado o curso ao qual ele pertence. O aluno deve ser inserido no conjunto geral de alunos do sistema e também no conjunto de alunos daquele curso;
- listar todos os alunos que não estão regulares, agrupando-os de acordo com seus respectivos cursos;

- listar os resultados dos cursos:
 - se receber um número inteiro entre 1 e 5, deve exibir todos os cursos que possuem aquele Conceito ENADE;
 - se receber uma *String* contendo o nome de uma área, deve exibir os cursos pertencentes àquela área e seus respectivos conceitos. Os cursos devem ser apresentados em ordem decrescente de conceito.

As demais funcionalidades serão especificadas nas próximas etapas do sistema.

6 O diagrama de classes UML

Nesta primeira atividade, você deve criar **apenas** o diagrama de classes UML do sistema descrito neste documento, contendo todas as **classes**, **atributos**, **métodos** e **relacionamentos** apropriados entre as classes.

É importante que você utilize corretamente, sempre que pertinente, as boas práticas de orientação a objetos, como **herança**, **reescrita de métodos**, **sobrecarga de métodos**, **encapsulamento**, reuso de código, etc. DICA: é **preciso** utilizar **herança**, **reescrita** e **sobrecarga** neste sistema.

Não é necessário incluir os construtores, métodos *get*, métodos *set* ou métodos *toString* no diagrama. Também não é necessário se preocupar com a leitura dos dados por enquanto.

Você pode utilizar programas específicos para modelagem OO, editores de imagens ou até mesmo desenhá-lo num papel. Deve ser enviado apenas um arquivo único contendo o diagrama completo (pdf, jpeg ou png).

7 Observações

1. Esta atividade vale 20 pontos e deve ser entregue até 30/09.
2. Deve ser enviado um único arquivo com a imagem do diagrama de classes UML do sistema, considerando as boas práticas de POO.
3. A atividade pode ser feita em grupos de até **dois** integrantes. Lembre-se de identificar os integrantes do grupo.
4. Atividades entregues após o prazo serão automaticamente rejeitadas.
5. Atividades consideradas plágio terão nota 0 para quem copiou e para quem forneceu a atividade, e serão enviadas ao Conselho de Ética.
6. A atividade deve ser enviada na sala da disciplina do AVA.
7. Em caso de dúvidas na especificação da atividade ou na própria atividade, contate-me em sala de aula.